2026.03.31

우리FISA 클라우드 서비스 개발반 백엔드 세미나

대용량 트래픽 대응하기

남인서

목차

01 테스트 시나리오 & 환경

02 테스트 과정

03 결론

Q. 좋은 서버란?

Q. 좋은 서버란?

- 고성능 - 빠른 응답, 낮은 지연
- 고효율 - 자원을 효율적으로 사용
- 안정성 - 부하 증가에도 무너지지 않음
- 일관성 - 항상 같은 결과를 보장
- 보안성

...

정말 제대로 동작하고 있을까?

어디까지 버틸 수 있을까?

→ 대용량 트래픽 실험

02

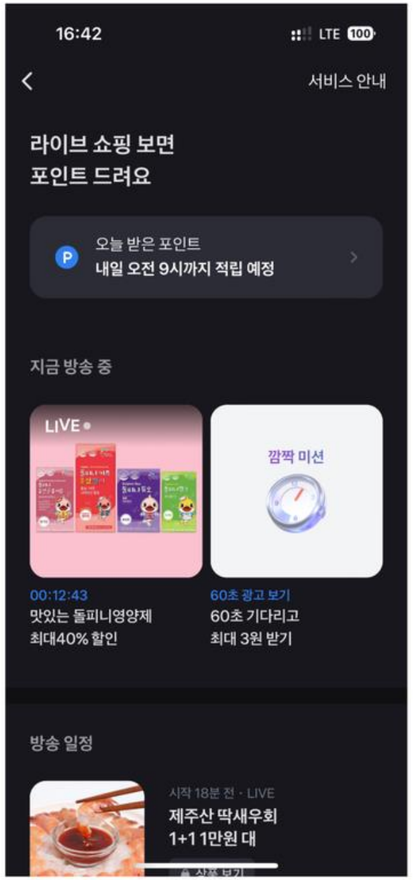
테스트 시나리오

toss tech

Engineering Design Product 구독하기 채용 바로가기

서버 증설 없이 처리하는 대규모 트래픽

이렇게 라이브 쇼핑 보기 서비스는 피크 시간대 동시 접속자 수는 분당 수십만 명, 포인트 지급 요청 API 요청은 초당 수십만 건이 오는 서비스로 성장했는데요. 급격히 늘어난 트래픽은 성장하는 서비스 서버에 치명적일 수 있어요. 서버가 트래픽을 유연하게 처리하지 못하면 유저에게 안 좋은 경험을 제공하고, 최악의 상황에서는 이탈할 수도 있기 때문이에요.



02

테스트 시나리오

포인트 지급 요청 기능을 개발할 때는 아래 네 가지 요소를 반드시 고려해야 했습니다.

- 한 유저에게 포인트가 중복 지급돼서는 안됩니다.
- 유저가 포인트가 지급되었다는 걸 즉시 인지할 수 있어야 합니다.
- 포인트가 지급되었다는 걸 토스의 포인트 지급 내역 원장에 기록할 수 있어야 합니다.
- 선착순에 들지 못하면 포인트 지급을 하면 안 됩니다.

비즈니스 요구사항:

사용자가 포인트 지급 요청을 하면
1.중복 지급인가? (DUPLICATED)
2.선착순에 들었는가? (SOLD_OUT)
확인 후 지급,

지급 시 원장에 기록 (SUCCESS)

02

테스트 시나리오

포인트 지급 API로 요청이 단시간에 몰리는 상황

```
measure: {
  executor: 'constant-arrival-rate',
  rate: TARGET_RPS,
  timeUnit: '1s',
  duration: '60s',
```

목표 RPS(Requests per second) = 10,000

→ 10,000 * 60s = 600,000 requests

→ 최대 포인트 수 = **100,000**

```
thresholds: {
  'http_req_duration{phase:measure}': ['p(95)<300'],
  'error_rate{phase:measure}':        ['rate<0.01'],
},
```

테스트 성공 기준:

- P95 Latency < 300ms
- Error rate < 0.01

02

기술 스택

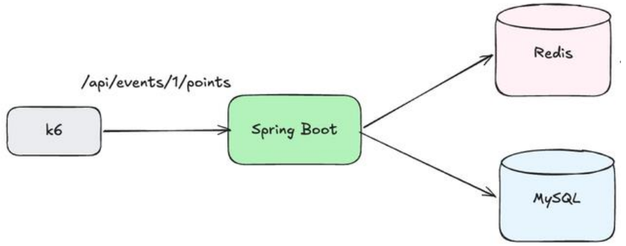
03

서버 발전 단계

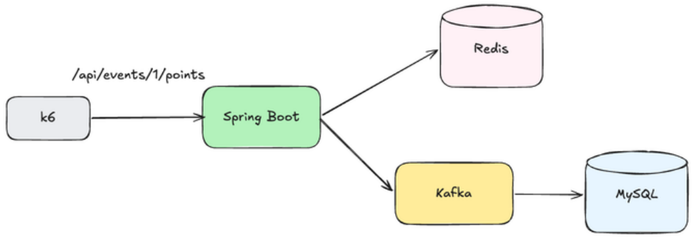
단계별로 더 많은 요청을 처리할 수 있지 않을까?

Phase 1

Phase 2

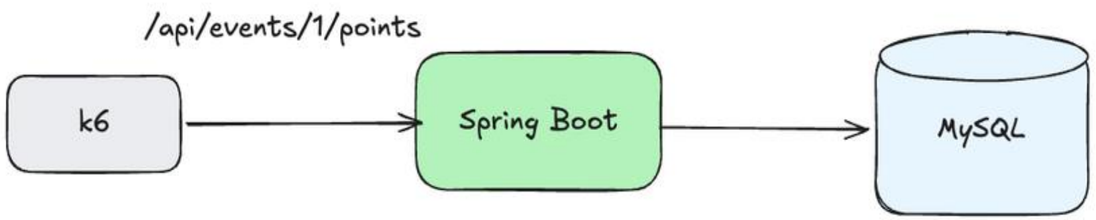


Phase 3



03

Phase 1: Baseline



기본 구조:

DUPLICATED → DB read

SOLD_OUT → DB read

SUCCESS → DB write

```
@Transactional // 하나의 트랜잭션으로 묶기  1 usage  sene03
public ClaimResult claim(String eventId, String userId) {
    if (pointLedgerRepository.existsByEventIdAndUserId(eventId, userId)) {
        return ClaimResult.DUPLICATE;
    }

    long currentCount = pointLedgerRepository.countByEventId(eventId);
    if (currentCount >= maxClaims) {
        return ClaimResult.SOLD_OUT;
    }

    pointLedgerRepository.save(new PointLedger(eventId, userId));
    return ClaimResult.SUCCESS;
}
```

03

Phase 1: Baseline

```yaml
hikari:
  maximum-pool-size: 10
  connection-timeout: 3000
```

```yaml
mysql:
  image: mysql:8.4.8
  deploy:
    resources:
      limits:
        cpus: '4.0'
```

```yaml
api:
  build: .
  deploy:
    resources:
      limits:
        cpus: '2.0'
```

테스트 설정값:

HikariCP

- connection timeout : 3s
- maximum pool size : 10

CPU:

- mysql : 4 core
- api (spring server) : 2 core

03

Phase 1: Baseline

	Latency (p95)	RPS
목표	≤ 300ms	10,000
Phase 1	3,139ms	2,118

Actuator threaddump

```
=== Tomcat Worker Threads ===
{
  "busy": 12,
  "max": 200,
  "running": 11,
  "waiting": 188,
  "blocked": 0
}

=== HikariCP ===
Active:  10.0
Pending: 188.0
Idle:    0.0
```

waiting thread 개수 188개

→ Hikari Pool connection 획득을 위해 대기

그럼 Connection Pool을 늘리면 되나?

03

Phase 1: Baseline

항목	pool=10	pool=30
measure RPS	2,118	**1,766**
avg waiting	1,725ms	**2,006ms**
p95 latency	3,139ms	**2,658ms**

실패!...

pool=30:

- 30개 커넥션이 동시에 DB 접근 → CPU 경합이 발생

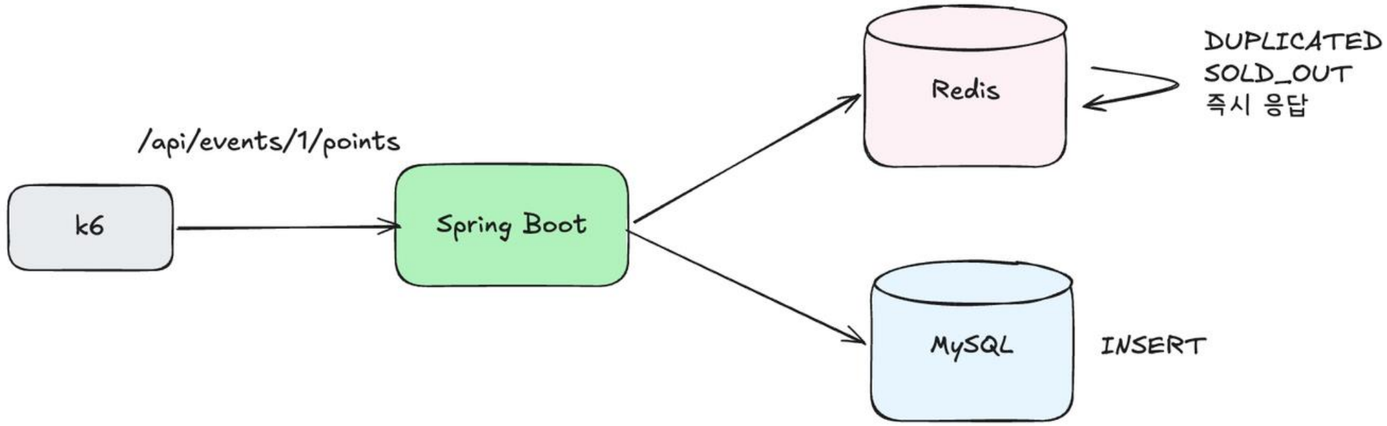
→ 쿼리당 실행 시간이 길어짐

connections = ((core_count * 2) + effective_spindle_count)

4*2 + 1 = 9

03

Phase 2: Redis



Redis 도입:

DUPLICATED → 즉시 응답

SOLD_OUT → 즉시 응답

SUCCESS → MySQL INSERT → 응답 (10만 건)

03

Phase 2: Redis

```
// Lua 스크립트: duplicate check → count check → increment → user 등록 (원자적 실행)
// KEYS[1] = event:{eventId}:count, KEYS[2] = event:{eventId}:users
// ARGV[1] = userId, ARGV[2] = maxClaims
private static final RedisScript<String> CLAIM_SCRIPT = RedisScript.of( script: """
        if redis.call('SISMEMBER', KEYS[2], ARGV[1]) == 1 then
          return 'DUPLICATE'
        end
        local count = tonumber(redis.call('GET', KEYS[1]) or '0')
        if count >= tonumber(ARGV[2]) then
          return 'SOLD_OUT'
        end
        redis.call('INCR', KEYS[1])
        redis.call('SADD', KEYS[2], ARGV[1])
        return 'SUCCESS'
        """, String.class);
```

event:{eventId}:count → 횟수

event:{eventId}:users → 받은 사용자 set

03

Phase 2: Redis

	Latency (p95)	RPS
목표	≤ 300ms	10,000
Phase 1	3,139ms	2,118
Phase 1 (pool=30)	2,658ms	1,766
Phase 2 (Redis)	1,369ms	6,850

→ 개선되긴 했지만 아직 멀었다!

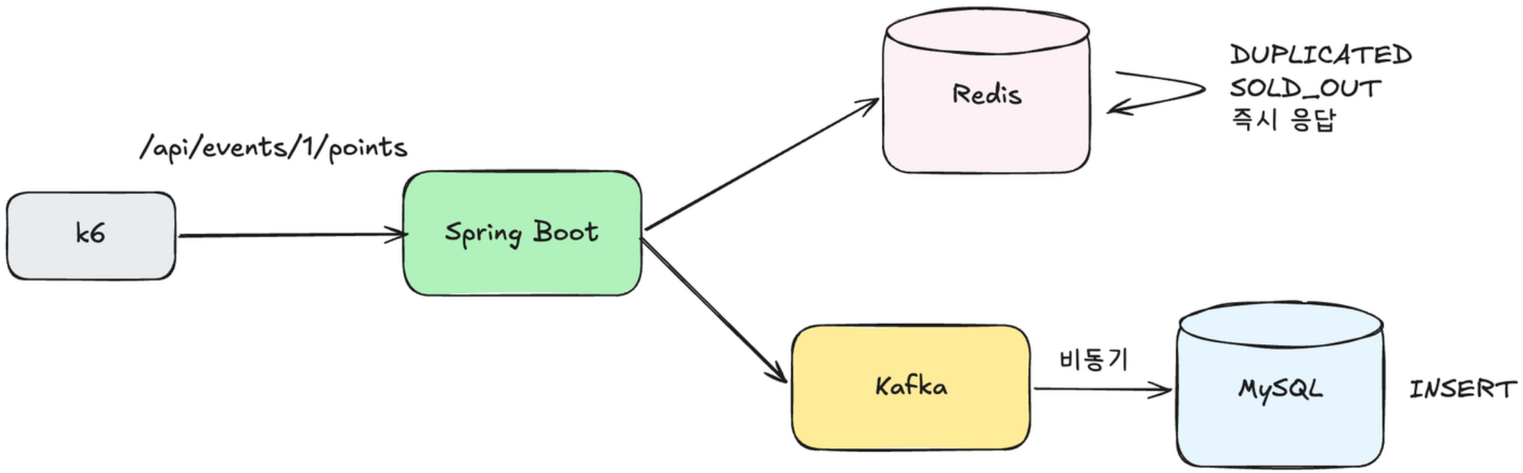
```
=== Tomcat Worker Threads ===
{
  "busy": 2,
  "max": 200,
  "running": 2,
  "waiting": 198,
  "blocked": 0
}

=== HikariCP ===
Active:  0.0
Pending: 0.0
Idle:    10.0
```

포인트가 소진된 후 DB I/O가 발생하지 않는 모습

03

Phase 3: Kafka



Kafka 도입:

DUPLICATED/SOLD_OUT → Redis 응답

SUCCESS → Kafka publish → 이후 Consumer가 INSERT

03

Phase 3: Kafka

#	목표 RPS	measured RPS	Latency(P95)	k6 timeout(10s)
1	10,000	**9,882**	208ms	**0건**
2	12,000	**11,945**	136ms	**0건**
3	13,000	4,214	2,040ms	188건(0.06%)
4	15,000	4,337	2,222ms	459건(0.15%)

RPS ~12,000 Latency < 300ms
목표는 달성함!

그 이상의 부하를 주면 왜 latency가 급증하는가?

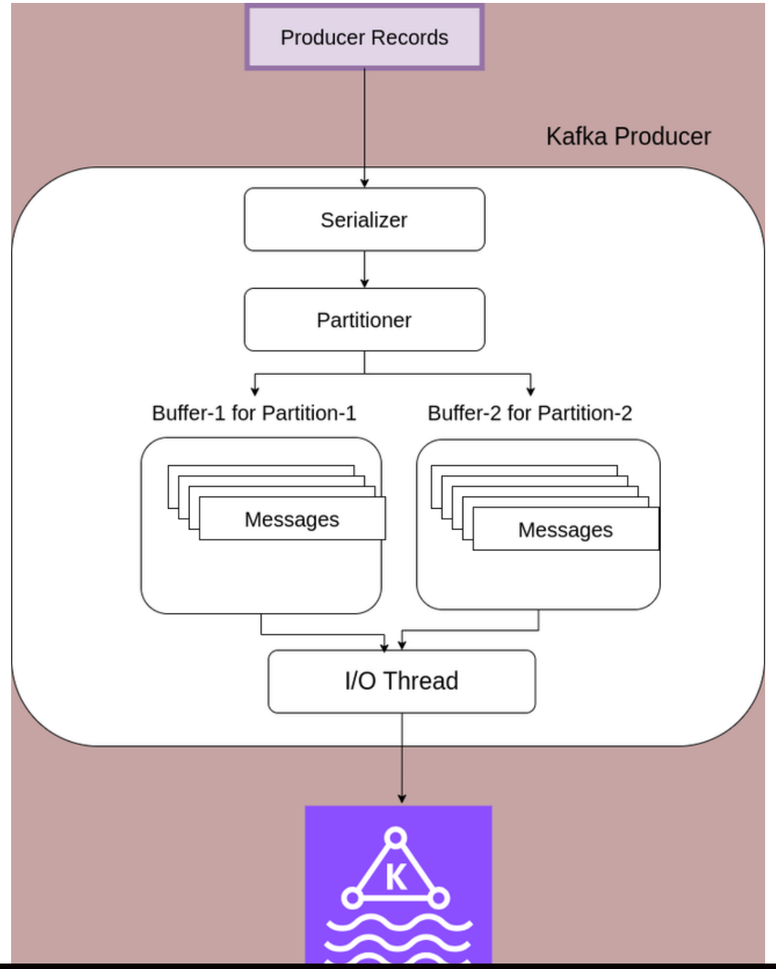
03

Phase 3: Kafka

추론 원인: Kafka 버퍼 포화

Kafka Partition 버퍼 사이즈 기본 32MB
버퍼 포화 → 블로킹 시작
→ k6 Timeout(10s) 발생

그럼 Partition 개수를 늘리면 해결되나?




```
=== Thread dump @ 35s (measure phase) ===
Active http-nio worker threads: 200
   73x  BLOCKED | org.apache.kafka.clients.producer.internals.RecordAccumulator.append
   40x  TIMED_WAITING | jdk.internal.misc.Unsafe.park
   36x  RUNNABLE | org.springframework.data.redis.serializer.StringRedisSerializer.deserialize
```

03

Phase 3: Kafka

항목	단일 파티션	3 파티션
measure_rps	**11,945**	**3,947** (-67%)
p95 (ms)	136	1,851
avg waiting	45ms	1,168ms
timeouts	0	192

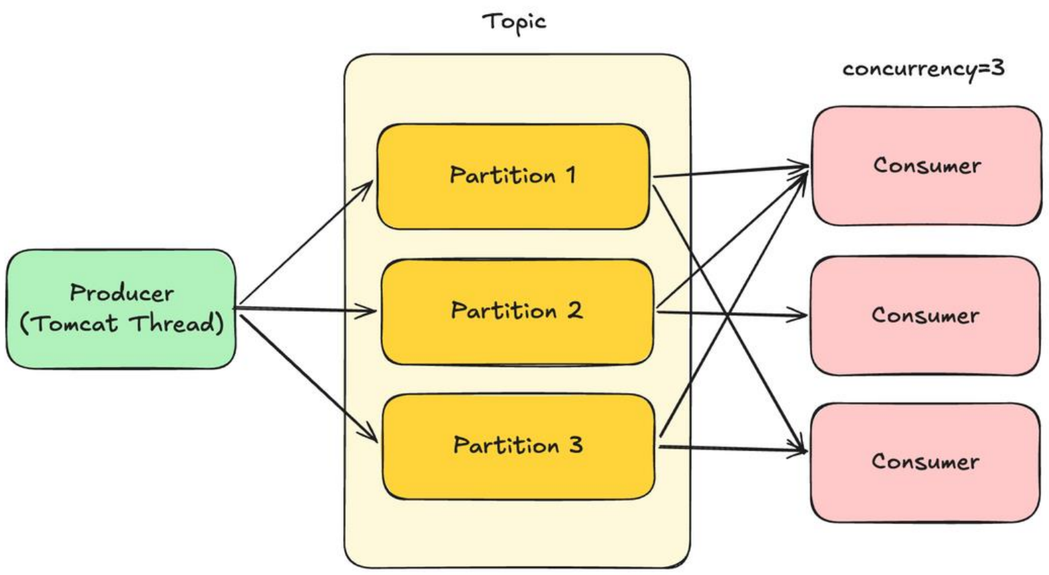
RPS=12000으로 실행한 결과 비교

→ 상황 악화

03

Phase 3: Kafka

무엇이 문제였을까?



버퍼 소비를 빠르게 하기 위해서
Kafka consumer thread 3개로 늘렸던 상황

변인통제를 위해 CPU 2core로 제한

```yaml
api:
  build: .
  deploy:
    resources:
      limits:
        cpus: '2.0'
```

NAME	CPU %	MEM USAGE / LIMIT
load-test-demo-mysql-1	14.65%	449.5MiB / 2GiB
load-test-demo-kafka-1	130.17%	565.1MiB / 7.653GiB
load-test-demo-redis-1	11.76%	13.2MiB / 7.653GiB
load-test-demo-api-1	208.70%	629.6MiB / 2GiB

Partition 늘리기 전 CPU 점유율

03

Phase 3: Kafka

항목	단일 파티션	3파티션 CPU=2	3파티션 CPU=4
measure_rps	**11,945**	3,994	**11,755**
p95 (ms)	136	1,942	**129**
avg waiting	45ms	1,154ms	**30ms**
timeouts	0	25	**0**

이미 CPU는 포화 상태였음
consumer 스레드가 늘어나면서 한계 초과 → latency 폭발

→ CPU core 4개로 늘려서 해결

03

그 다음은?

더 시도해보고 싶었던 것들

- 서버 증설이 필요한 때가 언제일까? 판단
- Nginx + Server Scaling
- 이후 Redis/Kafka 포화 해결하는 방법
- 정합성 문제

...

04

결론

1.부하를 감당하려면 내부 동작 원리를 알아야 한다

병목이 어디에서 발생하는가? 이 스레드가 무슨 역할? 어디에서 waiting/blocked?

2. 두들겨 맞아봐야 안다

트래픽이 몰리면 서버가 예상치 못한 동작을 보일 수 있음
운영 중에 생길 문제를 미연에 방지하려면 부하테스트 필요

Thank you!